Frontend architecture: React for a backend dev

> ***How to read this file:*** *the first half is GENERAL React/frontend knowledge (transferable to any React app / job / interview) — written for a BACKEND engineer, so frontend jargon is explained as it appears. The second half (marked "WORKED EXAMPLE") is how the WCX repo instantiates it. Learn the general model; use the repo to make it concrete.* ***Trust status:*** *repo specifics — send-chain files (`QuestionInput.tsx` → `Copilot.tsx` → `useAgentChat.ts` → `Copilot.api.ts`), `appendTextDelta` / `textFromSegments` / `reconcileReasoningTool` (agentSegments.ts), reducer actions, and stop/resume/detach — verified against live code through 2026-07-28; re-grep symbols before citing line numbers. React fundamentals are stable, language-general knowledge (not repo-specific).* ***Related:*** *[[kb-streaming-pipeline]] (the receive/render half — how the SSE stream is consumed and rendered), [[kb-storage-model]] (the durable client transcript / Cosmos sessions), [[kb-agent-loop]] (what the frontend is talking to server-side).*

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

Deep principle in one line: *declarative UI* — you describe what the screen should look like for a given state, and the framework computes the DOM changes; you never manually mutate the UI. You write the destination, not the driving directions.

Why it exists / what it solves. Coming from backend, my instinct is imperative: "find the thing, change the thing." That's exactly how the jQuery era worked — `$("#count").text(newValue)` scattered everywhere — and it did NOT scale. The UI and the underlying data would drift out of sync: you'd update the number in three places and forget the fourth, and now the screen lies about the state. React's answer is a single equation: **UI = f(state)**. Make state the one source of truth, and *re-derive* the entire UI from it whenever state changes. You stop hand-patching the screen; you change the state and let the framework reconcile the pixels. One-way data flow (state flows down, events flow up) is what tames the complexity — there's exactly one direction to reason about. This is also *why* the immutability/new-reference obsession exists (§1.1): the framework decides "did state change?" by reference identity, so a fresh object is how you say "yes, re-derive."

Real-world connection beyond this repo. This isn't a React quirk — it's the dominant paradigm. React, Vue, Svelte, and modern Angular all run on "UI = f(state)," and the *same* idea jumped platforms: SwiftUI (iOS), Jetpack Compose (Android), and Flutter are all declarative-from-state. So the transferable core — **component + props + one-way flow + hooks** — is knowledge you carry into essentially any product with a UI, web or native, for the rest of your career. Why bother as a backend dev? Full-stack fluency: you can read and review frontend PRs instead of rubber-stamping them, and — the part I actually care about — you can see *where the real API boundary sits*, because you understand what the client genuinely needs from your endpoint versus what it computes itself.

What breaks without it (the failure modes to recognize).

- **Mutating state in place** (`arr.push(x)`, `obj.field = y`) → the reference doesn't change → no re-render → the screen silently shows stale data. The nastiest class of frontend bug: no error, just wrong pixels.
- **Calling instead of passing** — `onClick={fn()}` runs `fn` during render and hands React the *result*; `onClick={fn}` hands React the function to call later (§1.4). One character, completely different behavior.
- **Prop-drilling hell** — without Context, a value from the top has to be threaded as a prop through every intermediate component that doesn't even use it.
- **Fetching inside components** instead of an api-module → network calls scattered and tangled into views → untestable, unmaintainable (§1.9).
- **Missing memoization** on hot paths → janky, over-firing re-renders, brutal for streaming UIs where state updates fire many times a second (§1.10).

Transferable mental model (why this should feel like home). State is the source of truth; the UI is *derived*, not stored. Data flows down, events flow up. A component is a pure-ish function of its props + state — same input, same output. These are the SAME instincts backend already trained: dependency injection (a child receives behavior via a callback prop instead of hard-wiring it, §1.8), separation of concerns (dumb view vs smart container, §1.3), and a pure transition function folding events into state (a reducer *is* event-sourcing, §1.5). Frontend just points those instincts at the view layer.

PART 1 — GENERAL KNOWLEDGE (React for a backend dev)

1.1 What React is & the core mental model

React is a library for building UIs out of **components** — self-contained functions that return a description of what should appear on screen. The one idea that unlocks everything else: React is **declarative**. You don't write "find the DOM node, change its text" (that's *imperative*, like jQuery). You write a function that says "given this data, the UI looks like THIS," and React figures out the DOM edits. You describe the destination; React computes the diff to get there.

- **Component** = a function that takes inputs (**props**, below) and returns UI. Think of it as a pure-ish render function: `data → UI`. Reusable and composable like backend functions/classes.
- **Virtual DOM** = an in-memory tree of lightweight JS objects describing the UI. The real browser DOM is slow to mutate; touching it repeatedly is expensive. So React renders to the virtual DOM first (cheap), then...
- **Reconciliation** = React **diffs** the new virtual-DOM tree against the previous one and applies only the minimal set of real-DOM changes. This is the "compute the diff to the destination" step. (Backend analogy: like a migration tool diffing desired-state vs current-state and emitting only the ALTER statements, instead of you hand-writing them.)
- **Re-rendering** = React re-runs a component function to produce a fresh virtual-DOM subtree. It does this when the component's **state or props change**. "Render" here means "call the function and diff," NOT "repaint pixels" — most renders produce zero actual DOM edits because the diff is empty.

Why immutability & new object references matter (the backend-dev gotcha). React decides "did this change?" by a **reference (identity) comparison** — `oldValue === newValue`, i.e. "is it literally the same object in memory?" — NOT a deep field-by-field compare (too slow to do everywhere). Consequence: if you MUTATE an object/array in place (`arr.push(x)`, `obj.field = y`), the reference is unchanged, so `old === new` is `true`, and React thinks nothing changed and skips the re-render — a silent bug. The rule is: to signal a change, produce a **NEW object/array** (`[...arr, x]`, `{...obj, field: y}`). The `...` is the **spread operator** = "shallow-copy the old entries into a fresh container." New container = new reference = `old !== new` = React re-renders. This is why React code looks obsessively copy-on-write; it's the price of cheap change detection.

1.2 JSX, and .tsx vs .ts

JSX is the HTML-looking syntax inside component functions — `return <div className="box">{name}</div>`. It is NOT a string and NOT real HTML; it's **syntactic sugar** that a compiler rewrites into function calls: `React.createElement("div", {className:"box"}, name)`. Those `createElement` calls build the virtual-DOM objects from §1.1. Curly braces `{}` drop back into JS ("interpolate this expression here").

- `.tsx` = a TypeScript file that is ALLOWED to contain JSX. `.ts` = TypeScript with NO JSX (plain logic, types, API helpers, utilities). The `x` literally enables the JSX parser. Rule of thumb: files that render UI are `.tsx`; hooks/helpers/api-modules that hold logic are `.ts`.

1.3 Component composition (has-a, not is-a)

Backend OO leans on **inheritance** (`class Admin extends User`). React deliberately does NOT; it uses **composition** — components CONTAIN other components ("has-a"), assembled by nesting them in JSX. A `<Page>` renders a `<Sidebar>` and a `<Content>`; it doesn't inherit from them. The outer component decides *what goes inside* and *what data to hand down*.

A common split (a separation-of-concerns discipline, not a language feature):

- **Presentational / "dumb" component** — only renders UI from its props and reports user actions back up. Holds no business logic, doesn't know where data comes from. Highly reusable. (Think: a pure view.)
- **Container / "smart" component** — holds state, fetches data, decides logic, and passes results DOWN into dumb children. (Think: the controller/orchestrator.) Keeping these apart is the frontend echo of keeping your handler/service/repository layers distinct.

1.4 Props: one-way data flow, data-down / events-up

Props ("properties") are the inputs a parent passes to a child, written as JSX attributes: `<Child name="Bob" count={3} />`. Two hard rules that trip up backend devs:

- **Data flows ONE way: down (parent → child).** A child receives props as read-only inputs. It CANNOT reach up and grab or mutate the parent's state — there is no upward pointer. (Contrast a backend object graph where a child might hold a back-reference to its parent; React forbids that on purpose, so data has exactly one source of truth and one flow direction.)

- **Props carry data AND callbacks.** A prop's value can be a function, because **functions are first-class values** in JS. This is the key that makes one-way flow workable: if a child needs to affect the parent, the parent passes DOWN a function, and the child CALLS it — sending information back "up" by invocation. The pattern name is **data-down, events-up**.

The phone-number-vs-dialing distinction (memorize this). `onClick={handleClick}` — NO parentheses — passes the function itself, like handing someone a phone NUMBER to call later. `onClick={handleClick()}` — WITH parentheses — CALLS it right now during render and passes the *result*, like dialing the number immediately (almost always a bug). Passing (`fn`) vs invoking (`fn()`) is the single most common early React mistake.

Why can't the child just reach into the parent? Because that would create hidden two-way coupling and destroy the single-source-of-truth guarantee. The parent OWNS the state; the child gets a read-only copy (props) plus a callback to *request* changes. The parent stays in control of its own data.

1.5 State management: useState, useReducer, and why Context/Redux exist

State = data that belongs to a component and, when it changes, triggers a re-render. Two built-in tools:

- `useState` = one independent piece of state. `const [count, setCount] = useState(0)` gives you the current value and a setter; calling `setCount(1)` schedules a re-render with the new value. Reach for it when state is a few unrelated scalars.
- `useReducer` = ONE state value updated by a single `(state, action) → newState` function (the **reducer**), instead of many scattered setter calls. You call `dispatch({type: "INCREMENT", payload: 5})` ; the reducer receives the current state + that **action** (a plain object describing "what happened") and RETURNS the next state. It's the exact same shape as `Array.reduce` — you "reduce" a *stream of actions over time* into one evolving state value. Backend analogy: it's an **event-sourcing / command handler** — actions are commands, the reducer is the pure transition function, state is the fold of all commands so far.

When to reach for a reducer over useState: when the next state depends on the previous state in nontrivial ways, when several fields must change together atomically, or when update logic is complex enough that you want it in ONE pure, testable function instead of sprinkled across the component. Reducers MUST be pure and MUST return new references (§1.1) — never mutate the incoming state.

Context & Redux (why they exist): prop-drilling. Because data only flows down (§1.4), getting a value from a top component to a deeply nested one means threading it as a prop through EVERY intermediate component that doesn't even use it — tedious, noisy plumbing called **prop-drilling**. **Context** is React's built-in escape hatch: a provider makes a value available to any descendant without passing it hand-to-hand. **Redux** is an external library that centralizes ALL app state into one global store with one reducer, so any component can read/dispatch without drilling. Both solve the same pain (distant components sharing state); Context is lightweight/built-in, Redux is heavier with more structure. For local state, plain `useState / useReducer` is still the default — don't reach for globals prematurely.

1.6 Hooks: reusable stateful logic

Hooks are functions whose names start with `use*` (`useState` , `useReducer` , `useEffect` , `useContext` , ...). A hook is how a function-component "hooks into" React features like state and lifecycle. The deeper point for a backend dev: a hook **packages reusable stateful logic** so it can be shared across components WITHOUT inheritance or copy-paste — it's the composition mechanism for behavior, the way a mixin/trait/service would be on the backend.

- **Rules of hooks** (enforced, not stylistic): (1) only call hooks at the TOP LEVEL of a component or another hook — never inside loops, conditions, or nested functions; (2) only call them from React components or custom hooks. React tracks hooks by *call order* per render, so conditional calls would misalign that order and corrupt state. Just remember: hooks run unconditionally, every render, same order.
- **Custom hooks** = your own `use*` function that composes built-in hooks and RETURNS an object or functions — e.g. `useAgentChat()` returns `{ sendMessage, isLoading, messages }` . The component calls the hook and uses what it hands back. This is how you lift messy stateful logic OUT of a component into a reusable, independently-testable unit.

1.7 The 3 ways code crosses files (don't conflate them)

A recurring confusion for backend devs is "how did this function/value get into this file?" There are THREE distinct mechanisms, and only one of them is a "prop":

1. **PROPS** — parent → child, VIA JSX ONLY: `<Child onSend={sendQuestion} />` . Passes a value (data or a function) DOWN the component tree at render time. Direction and channel are fixed: down, through JSX.

2. **HOOK-RETURN** — a hook hands values BACK to its caller: `const chat = useAgentChat(...)` → `chat.sendMessage(...)`. Not a parent/child relationship; it's a function returning a bundle.
3. **IMPORT** — the ordinary module system: `import { agentChatApi } from "../Copilot.api"` → call it directly. Makes a NAME available inside a file at compile time.

`import` ≠ `prop`. Import makes a symbol *available in a file* (static, module-level, like a Java `import`/DI-container lookup). A prop *passes a value from a specific parent to a specific child instance* at runtime, down the tree. Confusing them leads to "why is this passed as a prop when I could just import it?" — see §1.8.

1.8 The dependency-injection parallel (this will feel like home)

When a child receives behavior via a **prop callback** instead of `import`ing the concrete function itself, that IS **dependency injection**. The child declares "I need something to call when the user clicks send" (an `onSend` prop) and the parent INJECTS the concrete implementation. The child stays decoupled and reusable — it works with ANY `onSend` the parent supplies, exactly like a service receiving a dependency through its constructor rather than `new`-ing it. So: `import` = the child hard-wires its own dependency; `prop callback` = the parent injects it. Prefer injection where you want the child generic/testable.

1.9 Data fetching patterns

Talking to a backend from React uses the browser's **`fetch`** API, which returns a **Promise** (JS's async result placeholder — resolves later with the response, or rejects on error). You consume it with **`async/await`** (`const res = await fetch(url); const data = await res.json();`) — syntactic sugar over promises that reads like sequential blocking code but is non-blocking under the hood (single-threaded event loop; `await` yields to it).

- **GET vs POST:** `fetch(url)` defaults to **GET** (read, no body, cacheable). To send data you pass options: `fetch(url, { method: "POST", body: JSON.stringify(payload), headers: {...} })`. Send = POST with a serialized body; the body must be a STRING (`JSON.stringify`), not a JS object.
- **Centralize API calls in an api-module.** Components should NOT call `fetch` directly. Put every network call in a dedicated module (e.g. `api.ts` / `*.api.ts`) exposing named functions. Why: one place for the base URL, auth headers, error handling, and retries; components import a clean `getUser()` instead of scattering URLs and fetch boilerplate; and it keeps components as pure view/orchestration (§1.3). Backend analogy: a repository/client layer instead of raw SQL in your controllers.

1.10 Rendering performance: memo, keys, unnecessary re-renders

Because a parent re-render re-renders its children by default (§1.1), a hot-path component can re-render far more than needed. Tools:

- **`React.memo(Component)`** wraps a component so it re-renders ONLY when its props actually change (by reference, §1.1) — memoization at the component level. A cheap guard around a component that renders often but whose inputs rarely change. (`useMemo` / `useCallback` are the same idea for values/functions: cache them across renders so their reference stays stable, so a memo'd child doesn't see "new" props every time.)
- **Keys.** When rendering a LIST (`items.map(...)`), each element needs a stable, unique `key` prop. React uses keys to match old and new list items during reconciliation (§1.1) so it can reorder/patch instead of destroying and rebuilding rows. Using array index as a key is a common bug when the list reorders. Missing/duplicate keys cause wrong renders and lost state. Memoization matters a LOT for streaming UIs, where state updates fire many times per second (§1.11) — without a memo guard, EVERY message re-renders on every token.

1.11 Consuming a stream in the browser

Two ways the browser reads a server stream (see [[kb-streaming-pipeline]] for the server side and the full receive/render walkthrough):

- **`EventSource` (the SSE API)** — `new EventSource(url)` opens a Server-Sent-Events connection, auto-parses `data:` frames, and auto-reconnects. Simple, but LIMITED: GET only, no custom request headers, no request body. Great for public one-way feeds; too restrictive once you need auth headers or a POST body.
- **`fetch` + `ReadableStream` reader** — call `fetch`, then read `response.body.getReader()` and loop `await reader.read()`, decoding bytes and hand-parsing frames yourself. More work, but you get full control: POST, custom headers, abort. This is what apps (including WCX) use when `EventSource`'s limits bite. The trade-off: YOU own the byte-buffering and frame-splitting the browser would otherwise do (see [[kb-streaming-pipeline]] §1.4 for framing).

PART 2 — WORKED EXAMPLE: the WCX send chain, reducer transcript, and stream control

The WCX Engineering Copilot frontend is a textbook instance of Part 1: a dumb input (§1.3) that reports events up via a prop callback (§1.4/§1.8), a container that orchestrates, a custom hook (§1.6) that owns the request/stream logic, a centralized api-module (§1.9), and a reducer (§1.5) holding the durable transcript. This file keeps the three easily-confused layers (component / hook / api-module) and the three ways functions cross files (§1.7) straight, and traces the one send path through them.

React fundamentals as they appear here (Day 6/11 notes, preserved)

- `useReducer` = single `(state, action)→newState` fn vs many `useState` calls; "reduce" a stream of actions into one state value.
- Parent-child = JSX nesting, composition ("has a") not inheritance. Outer decides what goes inside + what data to hand down.
- **Props = data + callbacks passed parent→child; one-way flow.** Child can't grab parent state; if it needs to affect the parent, the parent passes a callback. ≈ dependency injection (child receives behavior instead of importing it).
- `.tsx` allows JSX; without it you'd write `React.createElement("div", …)` .

Passing a function via props (the key fix)

A function is a VALUE (§1.4). `onSend={sendQuestion}` (NO parens) = PASSES the function into the child's `onSend` slot — does NOT call it. `onSend()` inside the child = NOW run it. Parent LENDS; child calls later (the phone-number-vs-dialing distinction, §1.4). Same function, two names: `sendQuestion` (parent, defined there) / `onSend` (child, the prop slot). Child is generic → reusability (DI, §1.8).

3 ways functions hook across files here (the §1.7 map, concrete)

- **PROPS** (down, parent→child, JSX only): `onSend={sendQuestion}` .
- **HOOK-RETURN:** `const agentChat = useAgentChat(...)` → hook returns `{sendMessage, …}` → parent calls `agentChat.sendMessage` . Hook (name starts `use`) = packaged reusable stateful logic.
- **IMPORT** (direct): `import { agentChatApi } from "../Copilot.api"` → call directly. NOT a prop.
- "import" ≠ "prop": import makes a name available in a file; prop passes a value parent→child.

Component types

`QuestionInput` = "dumb"/presentational (§1.3) — renders the input box, shouts "send!" via `onSend` , holds NO backend logic. Logic (build request, session/user info, call API) lives higher (Copilot → useAgentChat) = separation of concerns.

The send chain (the find-the-backend-call skill)

`QuestionInput.tsx` `onSubmit → onSend()` → `Copilot.tsx sendQuestion()` (reads `currentInput` from ITS OWN state, resolves agent name, builds NO prompt) → `useAgentChat.ts sendMessage()` (builds `IAgentRequest` , creates `ctx`) → `consumeAgentStream(ctx, convId, signal => agentChatApi(request, signal))` → `Copilot.api.ts agentChatApi` → `fetch("/copilot/agent/chat", {method:POST, body:JSON.stringify(options)})` → backend route.

- **Find-the-backend-call skill:** look for `fetch(` / a `*Api` fn → read the URL → check the method (`POST` =send, §1.9). This repo centralizes ALL fetch in `Copilot.api.ts` (agentChat/resume/feedback/session CRUD); components never fetch directly (§1.9). "Where's the backend call?" = "look in `*.api.ts` ".
- Request sent to SRE = only `{ message.content (raw text), conversationId, product, agent NAME }` . NO prompt/history/tools — those live server-side. See [[kb-agent-loop]].

ctx / StreamContext = a per-TURN record (not per-session)

A new `ctx` is created every `sendMessage` and discarded when that turn ends. It holds BOTH directions of one turn: `question` (outgoing), `agentState` (incoming response scratchpad — the live reply being streamed now), `allMessages` (a per-turn snapshot), `controller` (the AbortController), and `sessionId` / `streamKey` / `detached` .

SEND + RECEIVE are the SAME call (the deferred-callback / DI moment)

`consumeAgentStream(ctx, convId, (signal)=>agentChatApi(request, signal))` — argument 3 is a DEFERRED callback (a **thunk** = a wrapped, not-yet-run computation), NOT a chained call; `agentChatApi` is NOT invoked at that line (§1.4 pass-vs-call).

`consumeAgentStream` OWNS the flow: it creates the AbortController, RUNS the injected recipe (`makeRequest(controller.signal)` = where the fetch/SEND actually fires), then reads the SSE response off the same call. The SAME consumer is reused for resume by injecting a different maker (`agentResumeApi` → `/copilot/agent/resume`). This is dependency injection / inversion-of-control (§1.8): the consumer is generic; the caller injects "how to make the request." One call sends AND receives — there's no separate "now start listening" step to race.

Reducer state = the session-long client transcript (§1.5 here)

The reducer is `(state, action)→newState`. `CopilotState.messages: IChatMessage[]` is the on-screen transcript. It does NOT copy `ctx.allMessages` — it builds its OWN list via actions:

- `APPEND_MESSAGE` = `[...state.messages, payload]` (a NEW array, §1.1) = THE accumulation; fires per user msg + per assistant msg.
- `UPDATE_LAST_ASSISTANT` = swaps the last assistant message for an updated copy; fires per streamed chunk (this is what produces the typing effect — see [[kb-streaming-pipeline]]). `ctx.allMessages` (a per-turn copy) and `state.messages` (the durable client transcript) are fed the same messages INDEPENDENTLY — neither is derived from the other. `LOAD_SESSION` refills `state.messages` from the server on reopen (see [[kb-storage-model]]).

allMessages — purpose & snapshot timing

`ctx.allMessages` holds WHOLE TURNS, both roles (user + assistant interleaved) = the transcript — NOT LLM-only, NOT the fragments of one reply (those are `agentState.assistantSegments`, one scale down). Proof: the code SEARCHES for `role===ASSISTANT` (pointless if it were LLM-only). **Its purpose = input to `snapshotMessages` for the DETACH/background path, NOT normal render.** On the attached path, every chunk dispatches to the reducer and the screen renders from `state.messages`; `allMessages` is essentially dormant. The snapshot is captured at turn START → it EXCLUDES the reply being generated (that lives in `agentState`); `snapshotMessages` = frozen `allMessages` + the live `agentState` message stitched on top. The reply enters "history" only as the NEXT turn's frozen past.

STOP vs RESUME vs DETACH (don't conflate)

- **STOP** = `controller.abort()` via `signal` → kills the fetch. TERMINAL — no pause/resume.
- **RESUME** (`resumeStream` → `/agent/resume`) = reconnect to a STILL-GENERATING server turn after a PAGE RELOAD wiped client state; re-streams the current answer. (A live push channel can't replay history, which is why resume re-streams from the durable server side — cross-link [[kb-streaming-pipeline]].)
- **DETACH** (`detachStream`) = switch chats mid-stream → set `detached=true`, keep streaming into `agentState` but route output to the background store, and remove the controller from the abort-list so the chat-switch doesn't kill it. **REATTACH** = return to that chat → `detached=false`, re-add the controller, resume UI routing.
- Enables multiple NON-INTERFERING concurrent turns: each turn has its own `ctx` in a `Map` (keyed by `streamKey`), its own fetch/TCP stream, and its own controller → no shared state, no cross-talk. A same-chat second send is BLOCKED by `sendQuestion`'s `if (isLoading || effectiveStreaming) return`; cross-chat concurrency is allowed via detach.

Session APIs → Cosmos

`listSessionsApi` / `getSessionApi` / `deleteSessionApi` / `updateSessionApi` = the frontend side of Cosmos session METADATA (the sidebar catalog): list/rename/delete/pin → `fetch` → backend session store → Cosmos. See [[kb-storage-model]].

PART 3 — transferable takeaways (the interview/next-job version)

1. React is **declarative**: you write `data → UI`, React diffs the virtual DOM (**reconciliation**) and applies the minimal real-DOM edits.
2. Change detection is by **reference identity** (`old === new`), so **never mutate** — return NEW objects/arrays (spread `...`) to signal a change; this is why reducers and state updates are copy-on-write.
3. Data flows **one way (down)** as **props**; a child affects a parent only by CALLING a **callback prop** the parent passed down — **data-down, events-up**, which is just **dependency injection**.
4. Passing a function (`fn`) ≠ calling it (`fn()`) — the single most common early mistake (phone number vs dialing).
5. Three distinct ways code crosses files — **props** (down via JSX), **hook-return**, **import** — and `import` ≠ `prop`.
6. `useReducer` = one pure `(state, action)→newState` fold over a stream of actions (event-sourcing shape); reach for it over `useState` when updates are complex or must change together. Context/Redux exist to escape **prop-drilling**.

7. **Hooks** package reusable stateful logic (`use*` , called unconditionally at top level, in stable order); a custom hook returns the bundle a component consumes.
8. **Centralize `fetch` in an api-module** (POST = send a JSON-stringified body); keep components as dumb views / thin orchestrators.
9. On hot paths (streaming), guard with `React.memo` and give lists stable **keys**, or every item re-renders on every update.
10. Read a browser stream with `fetch` + a `ReadableStream` reader when you need POST/headers/abort; `EventSource` only when GET-only auto-SSE suffices.